Generalized pack declaration and usage

Document #:	P1858R2
Date:	2020-03-01
Project:	Programming Language C++
Audience:	EWG

Reply-to: Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Revision History

R1 [P1858R1] was presented in EWGI in Prague [EWGI.Prague]. An ambiguity in pack indexing was pointed out, which is resolved here (see pack indexing). The structured bindings simplification was split out into its own paper, [P2120R0], and this paper includes a new discussion on the merits of splitting it into multiple papers.

R0 [P1858R0] was presented in EWGI in Belfast [EWGI.Belfast], where further work was encouraged (9-4-1-0-0). Since then, several substantial changes have been made to this paper.

— The overloadable `operator...()` and `using ...` were removed.

— Pack expansion is now driven through structured bindings, which are extended on the library side as well.

— `constexpr` ranges removed from this paper, should be handled by expansion statements.

— Discussion of functions returning packs - no longer being proposed due to ambiguity of what it actually could mean.

— Discussion of pack literals.

§ 2 Introduction and Motivation

C++11 introduced variadic templates, one of the truly transformational language features introduced that standard. Despite pretty tight restrictions on where packs could be declared and how they could be used, this feature has proven incredibly successful. Three standards later, there hasn't even been much change. C++17 added a couple new ways to use packs (fold expressions [N4191] and using-declarations [P0195R2]), and C++20 will add a new way to introduce them (in lambda capture [P0780R2]). A proposal to iterate over them (expansion statements [P1306R1]) didn't quite make it. That's it.

There have been many papers in the interlude about trying to enhance pack functionality: a language typelist [N3728], fixed size and homogeneous packs [N4072] (and later [P1219R1]), indexing and slicing into packs [N4235] and [P0535R0], being able to declare packs in more places [P0341R0] and other places [P1061R0].

In short, there's been work in this space, although not all of these papers have been discussed by Evolution. Although, many of these have been received favorably and then never followed up on.

Yet the features that keep getting hinted at and requested again and again are still missing from our feature set:

1. the ability to declare a variable pack at class, namespace, or local scope
2. the ability to index into a pack
3. the ability to unpack a tuple, or tuple-like type, inline

All efficiently, from a compile time perspective. Instead, for (1) we have to use `std::tuple`, for (2) we have to use `std::get`, `std::tuple_element`, or, if we're only dealing with types, something like `mp_at_c` [Boost.Mp11], for (3) we have to use `std::apply()`, which necessarily introduces a new scope. `std::apply()` is actually worse than that, since it doesn't play well with callables that aren't objects and even worse if you want to use additional arguments on top of that:

```
std::tuple args(1, 2, 3);

// I want to call f with args... and then 4
std::apply([&](auto... vs){ return f(vs..., 4); }, args);
```

Matt Calabrese is working on a library facility to help address the shortcomings here [Calabrese.Argot].

This paper attempts to provide a solution to these problems, building on the work of prior paper authors. The goal of this paper is to provide a better implementations for a library `tuple` and `variant`, ones that ends up being much easier to implement, more compiler friendly, and more ergonomic. The paper will piecewise introduce the necessary language features, increasing in complexity as it goes, and is divided into two broad sections:

— introducing the ability to declare packs in more places and to index into them

— introducing the ability to convert types to packs and to index into them

§ 3 Packs, packs, packs

This section will propose several new language features, with the motivating example being a far simpler implementation of `tuple`, but which are generally applicable to all uses of variadic templates in C++ today.

§ 3.1 Member packs

This paper proposes the ability to declare a variable pack wherever we can declare a variable today:

```
namespace xstd {
    template <typename... Ts>
    struct tuple {
        Ts... elems;
    };
}
```

That gives us all the members that we need, using a syntax that arguably has obvious meaning to any reader familiar with C++ packs. All the usual rules follow directly from there. That class template is an aggregate, so we can use aggregate initialization:

```
xstd::tuple<int, int, int> x{1, 2, 3};
```

Or, in C++20:

```
xstd::tuple y{1, 2, 3};
```

§ 3.2 Empty variable pack

What does `xstd::tuple<> t;` mean here? The same way that an empty function parameter pack means a function taking no arguments, an empty member variable pack means no member variables. `xstd::tuple<>` is an empty type.

§ 3.3 Constructor and initializer packs

But `tuple` has constructors. `tuple` has *lots* of constructors. We're not going to go through all of them in this paper, just the interesting ones. But let's at least start with the easy ones:

```
namespace xstd {
    template <typename... Ts>
    class tuple {
    public:
        constexpr tuple() requires (std::default_constructible<Ts> && ...)
            : elems()...
        { }
```

```
        constexpr tuple(Ts const&... args)
                requires (std::copy_constructible<Ts> && ...)
            : elems(args)...
        { }

    private:
        Ts... elems;
    };
}
```

Let's pick a more complex constructor. A `std::tuple<Ts...>` can be constructed from a `std::pair<T, U>` if `sizeof...(Ts) == 2` and the two corresponding types are convertible. How would we implement that? To do that check, we need to get the corresponding types. How do we get the first and second types from `Ts`?

3.4 Pack Indexing

This paper proposes a "simple selection" facility similar to the one initially introduced in [N4235] (and favorably received in Urbana 2014): `T...[I]` is the `I`th element of the pack `T`, which is a type or value or template based on what kind of pack `T` is. Later sections of this paper will discuss why this paper diverges from that original proposal in choice of syntax.

Such indexing allows for implementing the pair converting constructor:

```
namespace xstd {
    template <typename... Ts>
    class tuple {
    public:
        template <std::convertible_to<Ts...[0]> T,
                  std::convertible_to<Ts...[1]> U>
            requires sizeof...(Ts) == 2
        constexpr tuple(std::pair<T, U> const& p)
            : elems...[0](p.first)
            , elems...[1](p.second)
        { }
    private:
        Ts... elems;
    };
}
```

Notably, in an earlier example we constructed the pack as a single entity and here we are constructing each element of the pack separately. Both are fine. We'll see a simpler way to do this later.

A properly constrained converting constructor from a pack:

```
namespace xstd {
    template <typename... Ts>
    class tuple {
    public:
        template <std::constructible<Ts>... Us>   // everything is convertible
            requires (sizeof...(Us) > 1 ||        // exclude the copy ctor match
                      !std::derived_from<std::remove_cvref_t<Us...[0]>, tuple>)
        constexpr tuple(Us&&... us)
            : elems(std::forward<Us>(us))...
        { }
    private:
        Ts... elems;
    };
}
```

As well as implementing `tuple_element`:

```
namespace std {
    template <size_t I, typename... Ts>
    struct tuple_element<I, xstd::tuple<Ts...>>
    {
        using type = Ts...[I];
    };
}
```

And a member `get`:

```
namespace xstd {
    template <typename... Ts>
    class tuple {
    public:
        template <size_t I>
        auto get() const& -> Ts...[I] const& {
            return elems...[I];
        }

    private:
        Ts... elems;
    };
}
```

This is a lot nicer than status quo. Consider just the implementation complexity difference between these two versions (where the status quo is taken from the libc++ implementation). I'm including just the pieces I've shown up 'til now: the default constructor, n-ary converting constructor, `pair` converting constructor, and the structured bindings opt-in as a member.

When considering the differences below, don't just look at the difference in number of lines. Consider also the difference in complexity. You almost don't even really have to think about how to implement the "proposed" version, whereas the status quo, you really have to plan quite carefully and think a lot at every step of the way.

Also I'm cheating a little and using Boost.Mp11 for pack indexing.

Status Quo	Proposed

`template <typename...> class tuple;`
 	
`template <typename... Ts>`
`struct std::tuple_size<tuple<Ts...>>`
`  : integral_constant<size_t, sizeof...`
`  (Ts)>`
`{ };`
 	
`template <size_t, typename... Ts>`
`struct std::tuple_element<I, tuple<Ts...>>`
`{`
`  using type = mp_at_c<I, mp_list<Ts...>>;`
`};`
 	
`template <size_t Idx, typename T>`
`class tuple_leaf {`
`  [[no_unique_address]] T value;`
`  template <typename...> friend class`
`  tuple;`
	`template <typename...> class tuple;`
	
	`template <typename... Ts>`
	`struct tuple_size<tuple<Ts...>>`
	`  : integral_constant<size_t, sizeof...`
	`  (Ts)>`
	`{ };`
	
	`template <size_t I, typename... Ts>`
	`struct tuple_element<I, tuple<Ts...>>`
	`{`
	`  using type = Ts...[I];`
	`};`
	
	`template <typename... Ts>`
	`class tuple {`
	`  [[no_unique_address]] Ts... elems;`
	
	`public:`

Status Quo

```
public:
    constexpr tuple_leaf()
        requires
        std::is_default_constructible<T>
      : value()
    { }

    template <typename U>
    constexpr tuple_leaf(std::in_place_t,
        U&& u)
        : value(std::forward<U>(u))
    { }
};

template <typename Indices, typename... Ts>
class tuple_impl;

template <size_t... Is, typename... Ts>
class tuple_impl<index_sequence<Is...>,
        Ts...>
    : public tuple_leaf<Is, Ts>...
{
public:
    tuple_impl() = default;

    template <std::constructible<Ts>... Us>
        requires (sizeof...(Us) > 1 ||
                !std::derived_from<

        std::remove_cvref_t<Us...[0]>,
                    tuple_impl>)
    constexpr tuple_impl(Us&&... us)
        : tuple_leaf<Is, Ts>(
            std::in_place, std::forward<Us>
        (us))...
    { }

    template <std::convertible_to<
                mp_at_c<0, mp_list<Ts...>>>
        T,
              std::convertible_to<
                mp_at_c<1, mp_list<Ts...>>>
        U>
        requires (sizeof...(Ts) == 2)
    constexpr tuple_impl(std::pair<T, U>
        const& p)
        : tuple_impl(p.first, p.second)
    { }
};

template <typename... Ts>
class tuple : public tuple_impl<
    make_index_sequence<sizeof...(Ts)>,
        Ts...>
{
public:
    using tuple::tuple_impl::tuple_impl;

    template <size_t I>
```

Proposed

```
    constexpr tuple()
        requires
        (std::default_constructible<Ts>
        && ...)
      : elems()...
    { }

    template <std::constructible<Ts>...
        Us>
        requires (sizeof...(Us) > 1 ||
                !std::derived_from<

        std::remove_cvref_t<Us...[0]>,
                    tuple>)
    constexpr tuple(Us&&... us)
        : elems(std::forward<Us>(us))...
    { }

    template <std::convertible_to<Ts...
        [0]> T,
              std::convertible_to<Ts...
        [1]> U>
        requires (sizeof...(Ts) == 2)
    constexpr tuple(std::pair<T, U>
        const& p)
        : elems...[0](p.first)
        , elems...[1](p.second)
    { }

    template <size_t I>
    constexpr auto get() const& -> Ts...
        [I] const& {
        return elems...[I];
    }
};
```

Status Quo	Proposed

constexpr auto get() ->
tuple_element_t<I, tuple> const& {
using leaf = tuple_leaf<I,
tuple_element_t<I, tuple>>;
return static_cast<leaf const&>
(*this).value;
}
};	

3.4.1 Pack Indexing Ambiguity

In EWGI session in Prague, James Touton pointed out the following example:

```
template <typename... Ts>
struct X {
    void f(Ts...[1]);
};
```

Since C++11, this is valid syntax. `X<int, char>::f` is a non-template member function that takes two arguments of types `int*` and `char*`. This would be more clear if we provided a name to the function parameter and instead spelled it `Ts... p[1]`.

With this paper, `Ts...[1]` would have different meaning: it would be the single type referring to the 2nd type in the pack `Ts`. In other words, `X<int, char>::f` would instead become a non-template member function that takes a single argument of type `char`.

This is a problem. Thankfully, it's a very small one. Neither gcc or msvc even allow this syntax today (clang and icc do), so that severely limits how much code exists in the wild that uses it. It's additionally unlikely to be used much since it's really just a very strange way to write `Ts*...` without any real benefit over this more direct formulation. Searching for `...[` on codesearch.isocpp finds 11 uses, all of which are in compiler test cases.

The proposed resolution is that `Ts...[1]` *always* means pack indexing. If a user really wants the C++11 meaning, they can either name the parameter (as in `Ts... p[1]`) or spell it as a pointer instead (as in `Ts*...`).

3.5 Extending Structured Bindings

See [P2120R0].

3.6 Packs at block scope

This section has thus far proposed the ability to declare member variable packs and member alias packs, which can greatly help implementing a type like `tuple`. For thoroughness, this paper also proposes the ability to declare variable packs and alias packs at block scope. That is:

```
template <typename... Ts>
void foo(Ts&&... ts) {
    using ...decayed = std::decay_t<Ts>;
    auto... copies = std::forward<Ts>(ts);
}
```

3.7 Implementing variant

The ability to declare a member pack and index into packs would also make it substantially easier to implement `variant`. In the same way that this paper is proposing allowing a member pack of a class type:

```
template <typename... Ts>
struct tuple {
    Ts... vals;
};
```

This paper also proposes allowing a member pack of a union type:

```
template <typename... Ts>
union variant {
    Ts... alts;
};
```

This doesn't just fall out naturally in the same way the class type case does. Packs behave as a single entity in the language today, and yet here we would need to have a part of an entity - we wouldn't initialize the pack `alts...`, we would necessarily only have to initialize exactly one of element of that pack. But I don't think that's a huge stretch. And it makes for a substantial improvement in how user-friendly the implementation is.

As with the earlier tuple example, with the proposed language changes, this is something you can just sit down and implement. It practically rolls off the page, because the language would let you express your intent better. With the status quo, this is just a lot more design work in order to eventually produce a lot more code that is harder to understand and takes longer to compile.

The following is an implementation solely of the default constructor, the destructor, and `get_if`:

Status Quo

```
template <size_t I, typename... Ts>
union impl { };

template <size_t I, typename T, typename...
        Ts>
union impl<I, T, Ts...>
{
public:
    template <typename... Args>
    constexpr impl(in_place_index_t<0>,
        Args&&... args)
        : head(std::forward<Args>(args)...)
    { }

    template <size_t J, typename... Args>
    constexpr impl(in_place_index_t<J>,
            Args&&... args)
        : tail(in_place_index<J-1>,
            std::forward<Args>(args)...)
    { }

    ~impl()
        requires
        std::is_trivially_destructible_v<T>
        = default;
    ~impl() { }

    auto get(in_place_index_t<0>) -> T& {
        return head;
```

Proposed

```
template <typename... Ts>
class variant {
    int index_;
    union {
        Ts... alts_;
    };
public:
    constexpr variant()
      requires
        std::default_constructible<Ts...[0]>
      : index_(0)
      , alts_...[0]()
    { }

    ~variant()
      requires
        (std::is_trivially_destructible_v<Ts>
        && ...)
      = default;
    ~variant() {
        mp_with_index<sizeof...(Ts)>(index_,
            [](auto I){
                std::destroy_at(&alts_...
        [I]);
            });
    }
};

template <size_t I, typename... Types>
```

Status Quo

```
    }
    template <size_t J>
    auto get(in_place_index_t<J>) -> auto& {
        return tail.get(in_place_index<J-1>);
    }
private:
    char _;
    T head;
    impl<I+1, Ts...> tail;
};

template <typename... Ts>
class variant {
    int index_;
    impl impl_;

public:
    constexpr variant()
        requires std::default_constructible<
            mp_at_c<0, mp_list<Ts...>>
        : index_(0)
        , impl_(in_place_index<0>)
    { }

    ~variant()
      requires
        (std::is_trivially_destructible_v<Ts>
        && ...)
      = default;
    ~variant() {
        mp_with_index<sizeof...(Ts)>(index_,
            [](auto I) {
                auto& alt =
        impl.get(in_place_index<I>);
                std::destroy_at(&alt);
            });
    }
};

template <size_t I, typename... Types>
constexpr variant_alternative_t<I,
        variant<Types...>>*
get_if(variant<Types...>* v) noexcept {
    if (v && v->index_ == I) {
        return &v-
        >impl.get(in_place_index<I>);
    } else {
        return nullptr;
    }
}
```

Proposed

```
constexpr variant_alternative_t<I,
        variant<Types...>>*
get_if(variant<Types...>* v) noexcept {
    if (v && v->index_ == I) {
        return &v->alts_...[I];
    } else {
        return nullptr;
    }
}
```

§ 3.8 Other Examples

§ 3.8.1 Enumerating over a pack

Pack indexing would also allow for the ability to enumerate over a pack, by iterating over the indices:

```
template <typename... Ts>
void enumerate(Ts... ts)
{
    template for (constexpr auto I : views::iota(0u, sizeof...(Ts))) {
        cout << I << ' ' << ts...[I] << '\n';
    }
}
```

3.8.2 `std::integer_sequence` and structured bindings

There is a paper in the pre-Cologne mailing specifically wanting to opt `std::integer_sequence` into expansion statements [P1789R0]. We could instead be able to opt it into structured bindings:

```
template <typename T, T... Values>
struct integer_sequence {
    using ...tuple_element = integral_constant<T, Values>;

    template <size_t I>
        requires (I < sizeof...(Values))
    constexpr auto get() const -> integral_constant<T, Values...[I]> {
        return {};
    }
};
```

We'll see an even more interesting use of this proposal's interaction with `index_sequence` in a later section.

4 Expanding a type into a pack

At this point, this paper has introduced:

— the ability to declare a packs in more places (member variable and member alias packs as well as block scope variable and alias packs)

— the ability to index into a pack using the `pack...[I]` syntax

— an extension of structured bindings to look for a specific member alias pack named `tuple_element`, to reduce the API footprint.

These, in of themselves, provide a lot of value. But I think there's another big step that we could take that could provide a lot of value. This paper has proposed the syntax `T...[I]` to be indexing into a pack, `T`. But there's a lot of similarity between a pack of types and a tuple - which is basically a pack of types in single type form. And while we can index into a tuple (that's what `tuple_element` does), the syntax difference between the two is rather large:

	Pack	Tuple
Types	`Types...[I]`	`tuple_element_t<I, Tuple>`
Values	`vals...[I]`	`std::get<I>(tuple)`

The problem is that we're used to having the thing we're indexing on the left and the index on the right, and in the tuple case - this is inverted. If we could turn the `tuple` into a pack of its constituents, we could use the pack indexing operation. [P1061R1], coupled with this paper thus far, would allow:

```
// structured bindings can now introduce a pack, so this is a
// tuple --> pack conversion
```

```
auto [...elems] = tuple;

// pack indexing as usual
auto first = elems...[0];
```

Note also that [P1045R1] would allow adding an `operator[]` to `tuple` such that `tuple[0]` actually works.

This paper proposes a more direct mechanism to accomplish this.

4.1 Generalized unpacking with `[:]`

This paper proposes a non-overloadable `[:]` operator which can be thought of as the "add a layer of packness" operator. The semantics of this operator are to unpack an object into the elements that would make up its respective structured binding declaration. For a `std::tuple`, this would mean calls to `std::get<0>()`, `std::get<1>()`, … For an aggregate, this would mean aliases into its members. For the `xstd::tuple` we defined earlier, this would be calls into the members `get<0>()`, `get<1>()`, `...`. For arrays, this would be the elements of the array.

Once we add a layer of packness, the entity becomes a pack - and so all the usual rules around interacting with packs apply.

	Pack	Tuple
Types	`Types...[I]`	`Tuple::[:]...[I]`
Values	`vals...[I]`	`tuple.[:]...[I]`

That's a fairly involved syntax for a simple and common operation, so this paper proposes the shorthand `Tuple::[I]` for that syntax. This is not a typo. The syntax for indexing into a pack (`Ts...[I]`) and the syntax for indexing into a type (`Ts::[I]`) are different and it's important that they are different. A proper discussion of the motivation for the differing syntax will follow, but the key is that these syntaxes be unambiguous.

	Pack	Tuple	Tuple, sugared
Types	`Types...[I]`	`Tuple::[:]...[I]`	`Tuple::[I]`
Values	`vals...[I]`	`tuple.[:]...[I]`	`tuple.[I]`

This means we can take our tuple implementation from the previous section:

```
namespace xstd {
    template <typename... Ts>
    class tuple {
        Ts... elems;
    public:
        tuple(Ts... ts) : elems(ts)... { }

        using ...tuple_element = Ts;

        template <size_t I> requires I < sizeof...(Ts)
        auto get() const& -> Ts...[I]& { return elems...[I]; }
    };
}
```

And that is already sufficient to write:

```
xstd::tuple vals(1, 2, 3);
// direct indexing
assert(vals.[0] + vals.[1] + vals.[2] == 6);

// direct unpacking
auto sum = (vals.[:] + ...);
assert(sum == 6);
```

Here `vals.[0]` is the first structured binding, which for this tuple-like type means `vals.get<0>()`. `vals.[:]` is the pack consisting of all of the structured bindings, which means the pack `{vals.get<0>(), vals.get<1>(), vals.get<2>()}`.

§ 4.2 Syntax-free unpacking?

The last example demonstrates the ability to turn the tuple `vals` into a pack by way of `vals.[:]`, at which point we can do any of the nice pack things with that entity (e.g. use a *fold-expression*, invoke a function, etc). Why do we need that though? Can't we just have:

```
auto sum = (vals + ...);
```

The problem is this direction would lead to ambiguities (as almost everything in this space inherently does):

```
template <typename T, typename... U>
void call_f(T t, U... us)
{
    // Is this intended to add 't' to each element in 'us'
    // or is intended to pairwise sum the tuple 't' and
    // the pack 'us'?
    f((t + us)...);
}
```

Both meanings are reasonable - so it's left up to the programmer to express their intent:

```
// This adds 't' to each element in 'us'
f((t + us)...);

// This adds each element in 't' to each element in 'us',
// if the two packs have different sizes, ill-formed
f((t.[:] + us)...);
```

§ 4.3 Other examples of unpacking

Arrays are the easiest kind of structured binding. With `[:]`, they would unpack into all of their elements - which have the same type. Note that `arr[i]` and `arr.[i]` mean the same thing, except that in the latter case the index must be a constant expression.

```
void bar(int, int);

int values[] = {42, 17};
bar(values.[:]...); // equivalent to bar(42, 17)
```

This directly allows unpacking types with all-public members as well:

```
struct X { int i, j; };
struct Y { int k, m; };
```

```
int sum(X x, Y y) {
    // equivalent to: return x.i + x.j + y.k * y.k + y.m * y.m
    return (x.[:] + ...) + ((y.[:] * y.[:]) + ...);
}
```

And because for such types, structured binding calls the bindings themselves aliases rather than new variables, this works with bitfields:

```
struct B {
    uint8_t i : 4;
    uint8_t j: 4;
};
B b{.i = 2, .j = 4};
assert(b.[0] + b.[1] == 6);
```

4.3.1 Fixed-size homogeneous pack

This also provides a direct solution to the fixed-size pack problem [N4072]:

```
template <typename T, int N>
class Vector {
public:
    // I want this to be constructible from exactly N T's. The type T[N]
    // expands directly into that
    Vector(T[N]::[:]... vals);
};
```

The type `Vector<int, 3>` will have a *non-template* constructor that takes three `int`s. This works because `int[3]` is a type that can be used with structured bindings (being an array), so `int[3]::[:]` is the pack of types that would be the types of the bindings (i.e. `{int, int, int}`). Which then expands into three `int`s.

Note that this behaves differently from the homogenous variadic function packs paper [P1219R1] (I'm not claiming one behavior is better than the other - just noting the difference):

```
template <typename T, int N>
class Vector2 {
public:
    // independently deduces each ts and requires that
    // they all deduce to T. As opposed to the previous
    // implementation which behaves as if the constructor
    // were not a template, just that it took N T's.
    Vector2(T... ts) requires (sizeof...(ts) == N);
};
```

For instance:

```
Vector<int, 2>  x('a', 2); // ok
Vector2<int, 2> y('a', 2); // ill-formed, deduction failure
```

4.3.2 Construct a `tuple` from a `pair`

We can also simplify the example we saw earlier where we were constructing a `tuple` from a `pair`:

```
namespace xstd {
    template <typename... Ts>
    class tuple {
    public:
```

```
        template <std::convertible_to<Ts>... Us>
        constexpr tuple(std::pair<Us...> const& p)
            : elems(p.[:])...
        { }
    private:
        Ts... elems;
    };
}
```

§ 4.3.3 Implementing `make_index_sequence` without a compiler builtin

You have always been able to implement a version of `make_index_sequence` that would decompose into a sequence of integral constants:

```
template <size_t N>
struct index_seq {
    template <size_t I>
    constexpr auto get() const -> std::integral_constant<size_t, I> {
        return {};
    }
};

namespace std {
    template <size_t N>
    struct tuple_size<index_seq<N>>
        : integral_constant<size_t, N>
    { };

    template <size_t I, size_t N>
    struct tuple_element<I, index_seq<N>> {
        using type = integral_constant<size_t, I>;
    };
}

// a is an integral_constant<size_t, 0>
// b is an integral_constant<size_t, 1>
auto [a, b] = index_seq<2>();
```

But this is actually totally useless because you need to know the number of identifiers to declare. With [P1061R1], this becomes useful since you can write

```
auto [...Is] = index_seq<N>();
```

and now you don't need to indirect through a lambda. With this proposal, it becomes more useful still since you don't even need the extra declaration. You can implement Boost.Mp11's `mp_with_index` in a one liner:

```
template <size_t N, typename F>
decltype(auto) with_index(size_t i, F fn)
{
    return array{
        +[](F& f) -> decltype(auto) { return f(index_seq<N>{}.[:]); }...
    }[i](fn);
}
```

§ 4.4 Generalizing slicing further

The reason the syntax `[:]` is chosen as the "add a layer of packness" operator is because it allows a further generlization. Similar to Python's syntax for slicing, this paper proposes to provide indexes on one side or the other

of the `:` to take just sections of the pack.

For instance, `T::[1:]` is all but the first element of the type. `T::[:-1]` is all but the last element of the type. `T::[2:3]` is a pack consisting of only the third element.

Such a feature would provide an easy way to write a `std::visit` that takes the variants first and the function last:

```
template <typename... Args>
constexpr decltype(auto) better_visit(Args&&... args) {
    return std::visit(
        // the function first
        std::forward<Args...[-1]>(args...[-1]),
        // all the variants next
        // note that both slices on both Args and args are necessary, otherwise
        // we end up with two packs of different sizes that need to get expanded
        std::forward<Args...[:-1]>(args...[:-1])...);
}
```

The seemingly excessive amount of dots is actually necessary. `args` is a pack, `args...` unpacks it, `args...[:-1]` adds a layer of pack on it again - so it again needs to be expanded. Hence, `args...[:-1]...` is a pack expansion consisting of all but the last element of the pack (which would be ill-formed for an empty pack).

It would also allow for a single-overload variadic fold:

```
template <typename F, typename Z, typename... Ts>
constexpr Z fold(F f, Z z, Ts... rest)
{
    if constexpr (sizeof...(rest) == 0) {
        return z;
    } else {
        // we need to invoke f on z and the first elem in rest...
        // and recurse, passing the rest of rest...
        return fold(f,
            f(z, rest...[0]),
            rest...[1:]...);

        // alternate formulation
        auto head = rest...[0];
        auto ...tail = rest...[1:];
        return fold(f, f(z, head), tail...);
    }
}
```

§ 4.5 Examples with Boost.Mp11

Boost.Mp11 works by treating any variadic class template as a type list and providing operations that just work. A common pattern in the implementation of many of the metafunctions is to indirect to a class template specialization to do the pattern matching on the pack. This paper provides a more direct way to implement many of the facilities.

We just need one helper that we will reuse every time (as opposed to each metafunction needing its own helper):

```
template <class L> struct pack_impl;
template <template <class...> class L, class... Ts>
struct pack_impl<L<Ts...>> {
    // a pack alias for the template arguments
    using ...tuple_element = Ts;

    // an alias template for the class template itself
```

```
    template <typename... Us> using apply = L<Us...>;
};

template <class L, class... Us>
using apply_pack_impl = typename pack_impl<L>::template apply<Us...>;
```

Boost.Mp11	This proposal

`template <class L> struct mp_front_impl;`
`template <template <class...> class L, class T,`
`        class... Ts>`
`struct mp_front_impl<L<T, Ts...>> {`
`    using type = T;`
`};`
 	
`template <class L>`
`using mp_front = typename mp_front_impl<L>::type;`
	`template <class L>`
	`using mp_front = pack_impl<L>::[0];`

`template <class L> struct mp_pop_front_impl;`
`template <template <class...> class L, class T,`
`        class... Ts>`
`struct mp_pop_front_impl<L<T, Ts...>> {`
`    using type = T;`
`};`
 	
`template <class L>`
`using mp_pop_front = typename`
`        mp_pop_front_impl<L>::type;`
	`template <class L>`
	`using mp_pop_front =`
	`        apply_pack_impl<`
	`    L, pack_impl<L>::[1:]...>;`

`// you get the idea`
`template <class L>`
`using mp_second = /* ... */;`
 	
`template <class L>`
`using mp_third = /* ... */;`
 	
`template <class L, class... Ts>`
`using mp_push_front = /* ... */;`
 	
`template <class L, class... Ts>`
`using mp_push_back = /* ... */;`
 	
`template <class L, class T>`
`using mp_replace_front = /* ... */;`
 	
`// ...`
	`template <class L>`
	`using mp_second = pack_impl<L>::`
	`        [1];`
	
	`template <class L>`
	`using mp_third = pack_impl<L>::[2];`
	
	`template <class L, class... Ts>`
	`using mp_push_front =`
	`        apply_pack_impl<`
	`    L, Ts..., pack_impl<L>::`
	`        [:]...>;`
	
	`template <class L, class... Ts>`
	`using mp_push_back =`
	`        apply_pack_impl<`
	`    L, pack_impl<L>::[:]...,`
	`        Ts...>;`
	
	`template <class L, class T>`
	`using mp_replace_front =`
	`        apply_pack_impl<`
	`    L, T, pack_impl<L>::[1:]...>;`

§ 5 Can functions return a pack?

The previous draft of this paper [P1858R0] also proposed the ability to have a function return a pack. This revision removes that part of the proposal. Consider:

```
void observe();
void consume(auto...);

template <typename... Ts>
struct copy_tuple {
    Ts... elems;

    Ts... get() const {
        observe();
        return elems;
    }
};

copy_tuple<X, Y, Z> tuple(x, y, z);

// #1
consume(tuple.get()...);

// #2
consume(tuple.get()...[0]);

// #3
consume(tuple.get()...[0], tuple.get()...[1]);
```

There are two questions we have to answer for each call to `consume` there:

1. How many times is `observe()` invoked?
2. How many times are the `X`, `Y`, and `Z` copied?

There are two models we can think about for how the `get()` function might work:

Language Tuple

`get()` behaves as if it returns a language tuple. That is, it's syntax sugar for:

```
tuple<Ts...> get() const {
    observe();
    return {elems...};
}
```

Except that it's already a pack. With this model, every call to `get()` calls `observe()` a single time and copies every element.

Pack of functions

`get()` behaves as if it declares a pack of functions. That is, it's syntax sugar for:

```
Ts...[0] get<0>() const {
    observe();
    return elems...[0];
}

Ts...[1] get<1>() const {
    observe();
    return elems...[1];
```

```
}

// etc.
```

With this model, each individual element access incurs a call to `observe()`, but only the desired element is copied.

Which is better?

The problem is, both models are reasonable mental models and yet both models lead to some jarringly strange results:

— If we consider the language tuple model, then the example `#3` - `consume(tuple.get()...[0], tuple.get()...[1])` - would end up copying every element in the tuple, twice. Six copies, to get two elements?

— If we consider the pack of functions mode, then the example `#1` - `consume(tuple.get()...)` - would end up calling `observe()` three times. But it only looks like we're invoking a single function?

I don't think either model is better than the other, so this paper punts on this problem entirely. There is no longer a proposal to allow functions returning packs.

6 Disambiguation

There are many aspects of this proposal that require careful disambiguation. This section goes through those cases in turn.

6.1 Disambiguating Dependent Packs

If we have a dependent name that's a pack, we need a way to disambiguate that it's actually a pack. From [Smith.Pack]:

> ```
> template<typename T> void call_f(T t) {
>   f(t.x ...)
> }
> ```
>
> Right now, this is ill-formed (no diagnostic required) because "t.x" does not contain an unexpanded parameter pack. But if we allow class members to be pack expansions, this code could be valid – we'd lose any syntactic mechanism to determine whether an expression contains an unexpanded pack. This is fatal to at least one implementation strategy for variadic templates.

We need some kind of disambiguation. Perhaps some sort of context-sensitive keyword like `pack`?

```
template <typename T>
    requires (sizeof...(T::pack tuple_element))
//                      ~~~^^^^^^~~~~~~~~~~~~~~
struct tuple_size<T>
    : integral_constant<size_t, sizeof...(T::pack tuple_element)>
    //                                  ~~~^^^^^^~~~~~~~~~~~~~~
{ };
```

Unfortunately that doesn't work because of the possibility of something like:

```
template<typename ...Ts> struct X { using ...types = Ts; };
template<typename MyX> void f() {
```

```
  using Fn = void(typename MyX::pack types ...);
}
```

Today that's declaring a function type that takes one argument of type `typename MyX::pack` named `types` and then varargs with the comma elided. If we make the comma mandatory (as [P1219R1] proposes to do), then that would open up our ability to use a context-sensitive `pack` here.

Otherwise, we would need a new keyword to make this happen, and `pack` seems entirely too pretty to make this work. One thing we could consider is `packname` - which has the pleasant feature of having the same number of letters in it as both `template` and `typename`. But this paper suggests going a different direction instead.

6.1.1 Pack introducers

As a brief aside, it's worth enumerating the places in the language where we can introduce a pack today - because they all have an important feature in common:

```
template <typename ...T> // template parameter pack
void f(T ...t)           // function parameter pack
{
   [...u=t]{};           // init-capture pack
}
```

What these have in common is that is that the `...` always *precedes* the name that it introduces as a pack.

6.1.2 Proposed disambigutation

To that end, an appropriate and consistent mechanism to disambiguate a dependent member pack might also be to use *preceding* ellipses (which still need to be separated by a space). That is:

```
template <typename T>
    requires (sizeof...(T::...tuple_element))
struct tuple_size<T>
    : integral_constant<size_t, sizeof...(T::...tuple_element)>
{ };
```

And once we disambiguated that `T::tuple_element` is a pack, we still need to expand it - which may also require trailing ellipses:

```
template <size_t I, typename T>
    requires (sizeof...(T::...tuple_element))
struct tuple_element<I, T>
{
    using type = T::...tuple_element...[I];
};
```

That's admittedly a lot of dots fairly close together. If that isn't acceptable, then we can introduce a new disambiguating keyword:

```
template <size_t I, typename T>
    requires (sizeof...(T::packname tuple_element))
struct tuple_element<I, T>
{
    using type = T::packname tuple_element...[I];
};
```

6.2 Disambiguating packs of tuples

The previous section showed how to write `apply` taking a single function and a single tuple. What if we generalized it to taking multiple tuples? How do we handle a pack of tuples?

It's at this point that it's worth taking a step back and talking about disambiguation and why this paper makes the syntax choices that it makes. We need to be able to differentiate between packs and tuples. The two concepts are very similar, and this paper seeks to make them much more similar, but we still need to differentiate between them. It's the pack of tuples case that really brings the ambiguity to light.

The rules this paper proposes, which have all been introduced at this point, are:

— `e.[:]` takes a *pack-like type* (or object of such) and adds a layer of packness to it, by way of structured bindings. It never is applied to an existing pack, and it is never used to disambiguate dependent member access.

— `e.[I]` never removes a layer of packness. It is picking the `I`th element of a pack-like type.

— `e...[I]` always removes a layer of packness. It is picking the `I`th element of a pack.

— `` `e. ...fdisambiguates dependent member access and identifiesfas a pack The space between the.and...` `` is required.

That is, `pack...[I]` and `tuple.[I]` are valid, `tuple...[I]` is an error, and `pack.[I]` would be applying `.[I]` to each element of the pack (and is itself still an unexpanded pack expression). Rule of thumb: you need `...`s if and only if you have a pack.

`e.[I]` is an equivalent shorthand for `e.[:]...[I]`.

This leads to clear meanings of each of the following. If we have a function template taking an argument `e` which has a member `f`, where the kind of `e` is specified by the columns of this table and the kind of `f` is specified by the rows:

	e is a Pack	**e is a Pack-like type**	**e is not expanded**
f is a Pack	`foo(e. ...f... ...);`	`foo(e.[:]. ...f... ...);`	`foo(e. ...f...);`
f is a Pack-like type	`foo(e.f.[:]... ...);`	`foo(e.[:].f.[:]... ...);`	`foo(e.f.[:]...);`
f is not expanded	`foo(e.f...);`	`foo(e.[:].f...);`	`foo(e.f);`

The only two valid cells in that table in C++20 are the bottom-left and bottom-right ones. Note that every cell has different syntax, by design.

§ 6.3 Nested pack expansions

In order for the above table to work at all, we also need a new kind of pack expansion. When C++11 introduced pack expansion, the rules were very simple: The expression in `expr...` must contain at least one unexpanded pack expression and every unexpanded pack expression must have the same length.

But with the concepts introduced in this proposal, we have the ability to introduce new things that behave like unexpanded pack expressions within an unexpanded pack expression and we need to define rules for that. Consider:

```
template <typename... Ts>
void foo(Ts... e) {
    bar(e.[:]... ...);
}
```

```
// what does this do?
foo(xstd::tuple{1}, xstd::tuple{2, 3});
```

Following the rules presented above, `e.[:]` adds a layer of packness to each element in the pack (which is fine because `xstd::tuple`s are structured- bindable). But what then do the `...`s refer to?

We say that adding layer of packness in the middle of an existing unexpanded pack expression will hang a new, nested unexpanded pack expression onto that.

In the above example, `e` is an unexpanded pack expression. `e.[:]` is a nested unexpanded pack expression underneath `e`.

When we encounter the the first `...`, we say that it expands the most nested unexpanded pack expression that of the expression that it refers to. The most nested unexpanded pack expression here is `e.[:]`, which transforms the expression into:

```
bar((e.[0], e.[1], e.[2], /* etc. */, e.[M-1])...);
```

This isn't really valid C++ code (or, worse, it actually is valid but would use the comma operator rather than having `M` arguments). But the idea is we now have one more `...` which now has a single unexpanded pack expression to be expanded, which is the unexpanded pack expression that expands each element in a pack-like type.

A different way of looking at is the outer-most `...` expands the outer-most unexpanded pack expression, keeping the inner ones in tact. If we only touch the outer-most `...`, we end up with the following transformation:

```
bar(e_0.[:]..., e_1.[:]..., /* etc. */, e_N-1.[:]...);
```

The two interpretations are isomorphic, though the latter is likely easier to understand.

Either way, the full answer to what does `foo(xstd::tuple{1}, xstd::tuple{2, 3})` do in this example is that it calls `bar(1, 2, 3)`.

For more concrete examples, here is a generalized `apply()` which can take many tuples and expand them in order:

```
template <typename F, typename... Tuples>
constexpr decltype(auto) apply(F&& f, Tuples&&... tuples) {
    return std::invoke(
        std::forward<F>(f),
        std::forward<Tuples>(tuples).[:]... ...);
}
```

which, again more concretely, expands into something like:

```
template <typename F, typename T_0, typename T_1, ..., typename T_N-1>
constexpr decltype(auto) apply(F&& f, T_0 t_0, T_1 t_1, ..., T_N-1 t_N-1) {
    return std::invoke(std::forward<F>(f),
        std::forward<T_0>(t_0).[:]...,
        std::forward<T_1>(t_1).[:]...,
        ...
        std::forward<T_N-1>(t_N-1).[:]...);
}
```

And then we unpack each of these `...`s through the appropriate structured bindings rules.

Similarly, `tuple_cat` would be:

```
template <typename... Tuples>
constexpr std::tuple<Tuples.[:]... ...> tuple_cat(Tuples&&... tuples) {
    return {std::forward<Tuples>(tuples).[:]... ...};
}
```

And itself leads to a different implementation of generalized `apply`:

```
template <typename F, typename... Tuples>
constexpr decltype(auto) apply(F&& f, Tuples&&... tuples) {
    return std::invoke(
        std::forward<F>(f),
        tuple_cat(std::forward<Tuples>(tuples)...).[:]...
}
```

Admittedly, six `.`s is a little cryptic. But is it any worse than the current implementation?

§ 7 What about Reflection?

Two recent reflection papers ([P1240R0] and [P1717R0]) provide solutions for some of the problems this paper is attempting to solve. What follows is my best attempt to compare the reflection solutions to the generalized pack solutions presented here. I am not entirely sure about the examples on the left, but hopefully they are at least close enough to correct to be able to evaluate the differences.

Note that one notable example missing here is a constructor for `tuple` - I really don't know how to implement any of those constructors on top of reflection.

Reflection	This proposal

`// member pack declaration (P1717)`
`template <typename... Types>`
`class tuple {`
`  consteval {`
`    int counter = 0;`
`    for... (meta::info type : reflexpr(Types)) {`
`      auto fragment = __fragment struct {`
`        typename(type) unqualid("element_",`
`        counter);`
`      };`
`      -> fragment;`
`      ++counter;`
`    }`
`  }`
`};`
	`template <typename... Types>`
	`class tuple {`
	`  Types... element;`
	`};`

`// pack indexing (P1240)`
`template <size_t I, typename... Ts>`
`using at = typename(std::vector{reflexpr(Ts)...}`
`        [I]);`
	`template <size_t I, typename... Ts>`
	`using at = Ts...[I];`

`// generalized pack indexing (P1240)`
`template <typename... Types>`
`struct tuple {`
`  consteval static auto types() {`
`    return std::vector{reflexpr(Types)...};`
`  }`
	`template <typename... Types>`
	`struct tuple {`
	`    using ... = Types;`
	`};`

Reflection	This proposal

`};`
 	
`template <size_t I, typename T>`
`using tuple_element_t = typename(T::types()[I]);`
	`template <size_t I, typename T>`
	`using tuple_element_t = T.[I];`

`template <typename... Types>`
`struct tuple {`
`  consteval auto members() const {`
`  // return some range of vector<meta::info>`
`  // here that represents the data members.`
`  // I am not sure how to implement that`
`  }`
`};`
 	
`template <typename Tuple>`
`void call_f(Tuple const& t) {`
`  f(t.unreflexpr(t.members())...);`
`}`
	`template <typename... Types>`
	`struct tuple {`
	`  operator Types const&...() const& {`
	`  return elems; }`
	
	`  Types... elems;`
	`};`
	
	`template <typename Tuple>`
	`void call_f(Tuple const& t) {`
	`  return f(t.[:]...);`
	`}`

It's not that I think that the reflection direction is bad, or isn't useful. Far from. It's just that dealing with `tuple` is, in no small part, an ergonomics problem and I don't think any reflection proposal that I've seen so far can adequately address that: neither from the perspective of declaring a pack (as in for `tuple` or `variant`) nor from the perspective of unpacking a tuple into a function or other expression.

If reflection can produce something much closer to what is being proposed here, I would happily table this proposal. But it seems to me that it's fairly far off, and the functionality presented herein would be very useful.

§ 8 What about `std::pair`?

As shocking as it might be to hear, there are in fact other types in the standard library that are neither `std::tuple<Ts...>` nor `std::variant<Ts...>`. We should probably consider how to fit those other types into this new world.

The nice thing about implementing `tuple` with the new structured bindings direction is that because everything `tuple` is already a pack, staying in the pack world remains very easy. But `pair` doesn't have any packs, so it is missing out:

```
namespace xstd {
    template <typename T, typename U>
    struct pair {
        T first;
        U second;

        using ...tuple_element = ????;

        template <size_t I>
        auto get() const& -> tuple_element...[I] const&
        {
            if constexpr (I == 0) return first;
            else if constepxr (I == 1) return second;
        }
    };
}
```

How would we fill in the `????`s here? We need a pack there.

We could implement this in terms of tuple. `tuple<T, U>::[:]` is a pack of two types, `T` and `U`. This works directly with the proposal as presented. It's also a little odd, and indirect. But it works.

We could also come up with a way to introduce the pack we need directly, in-line, by way of a pack literal. Borrowing from the insight about how pack introducers work, this would either be `...<T, U>` (using the syntax from [P0341R0] with the extra preceding ellipsis) or `...{T, U}` (which would be in line with how gcc reports template errors when packs are involved).

A pack literal direction would also lead to a pack literal of values, which would add more fun with initialization:

```
auto    a = {1, 2, 3}; // a is a std::initializer_list<int>
auto... b = {1, 2, 3}; // b is a pack of int's
```

Those two declarations are very different. But also, they look different - one has `...` and the other does not. One looks like it is declaring an object and the other looks like it is declaring a pack.

Pack literals would also allow for adding default arguments to packs:

```
template <typename... Ts = ...<int>>
void foo(Ts... ts = ...{0});

foo(); // calls foo<int>(0);
```

I'm not sure if this is sufficiently motivated to pursue, but would be curious to hear what people think about this matter.

§ 9 One paper or many

R1 [P1858R1] of this paper included several features that build on each other:

1. Declaring packs in more places, and a disambiguation for dependent pack access.
2. An extension to the structured bindings protocol to simplify the opt-in by using a pack.
3. Indexing into a pack.
4. Turning a type into a pack.
5. Slicing a pack.

EWGI in Prague [EWGI.Prague], took two votes about separating these features:

1. Split the structured binding customization point changes into another paper? 3-8-5-1-1.
2. Split the indexing facilities into another paper? 2-2-9-3-1.

Many of the aspects of this proposal do seem separable. Indexing into a pack is nominally independent of declaring more packs, and both are nominally independent of turning a type into a pack. However, I think it's important to see the broad vision of what I want future pack facilities to look like - and it's especially important in this space to ensure that all the facilities properly interoperate. It's very easy to have issues with ambiguities here. All the syntax decisions were driven by the need to have every syntax look different from every other syntax in all contexts. Having all of the syntax-introducing proposals in a single paper ensures that they need to be considered as a cohesive whole and that they will be able to work together properly.

Of course, none of that argument applies to the structured bindings customization aspect of the initial proposal - it's a small, self contained improvement that has no ambiguity concerns and really only affects structured bindings. Which is why that part, and only that part, has been split off into its own proposal: [P2120R0].

10 Proposal

All the separate bits and pieces of this proposal have been presented one step at a time during the course of this paper. This section will formalize all the important notions.

10.1 Pack declarations

You can declare member variable packs, namespace-scope variable packs, and block-scope variable packs. You can declare alias packs. The initializer for a pack declaration has to be an unexpanded pack - which would then be expanded.

10.2 Dependent packs

Member packs and block scope packs can be directly unpacked when in non-dependent contexts. We know what they are.

In dependent contexts, anything that is not a pack must be explicitly identified as a pack in order to be treated as one. Similar to how we need the `typename` and `template` keyword in many places to identify that such and such an expression is a type or a template, a preceding `...` (or whatever alternate spelling) will identify the expression that follows it as a pack. If that entity is *not* a pack, then the indexing or unpacking expression is ill-formed.

Non-dependent packs do not need any disambiguation (the disambiguation is allowed, but unnecessary).

10.3 Pack Indexing

A pack can be indexed by expanding it and applying `[I]` to the result. `p...[0]` is the first element of the pack `p` (which can be a value, type, or template based on the kind of pack that `p` is).

This is a "sfinae-friendly" operation, so given:

```
template <typenae... Ts>
auto first(Ts... ts) -> Ts...[0] {
    return ts...[0];
}
```

`first()` triggers a substitution failure, rather than a hard error.

10.4 Adding a layer of packness

Given a value of a type that can be used on the right-hand side of a structured binding declaration, the `.[:]` operator may be applied to "add a layer of packness" - turning it into a pack of values consisting of what the structured bindings would have been.

A type that can be used in structured bindings can have the `::[:]` operator applied to it to likewise turn it into a pack of types.

```
void f(std::tuple<int, char, double> t) {
    // equivalent to g(std::get<0>(t), std::get<1>(t), std::get<2>(t))
```

```
    // or, possibly, std::apply(g, t)
    g(t.[:]...);

    // decltype(u) is the same as T - just a really complex way to
    // get there
    using T = decltype(t);
    std::tuple<T::[:]...> u = t;
}
```

The syntaxes `v.[I]` and `T::[I]` are shorthand for the syntaxes `v.[:]...[I]` and `T::[:]...[I]`, respectively.

10.5 Pack slicing

While `.[:]` and `::[:]` add a layer of packness, turning the expression into a pack consisting of all of the underlying elements, the slice notation can also include either a start or end index, or both, to get just part of the pack.

```
void h(std::tuple<int, char, double> t) {
    // a is a tuple<int, char, double>
    auto a = std::tuple(t.[:]...);

    // b is a tuple<char, double>
    auto b = std::tuple(t.[1:]...);

    // c is a tuple<int, char>
    auto c = std::tuple(t.[:-1]...);

    // d is a tuple<char>
    auto d = std::tuple(t.[1:2]...);
}
```

11 Acknowledgments

This paper would not exist without many thorough conversations with Agustín Bergé, Matt Calabrese, and Richard Smith. Thank you.

Thank you to David Stone for pointing out many issues.

12 References

[Boost.Mp11] Peter Dimov. 2017. Boost.Mp11: A C++11 metaprogramming library - 1.70.0.
https://www.boost.org/doc/libs/1_70_0/libs/mp11/doc/html/mp11.html

[Calabrese.Argot] Matt Calabrese. 2018. C++Now 2018: Argot: Simplifying Variants, Tuples, and Futures.
https://www.youtube.com/watch?v=pKVCB_Bzalk

[EWGI.Belfast] EWGI. 2019. EWGI Discussion of P1858R0.
http://wiki.edg.com/bin/view/Wg21belfast/P1858

[EWGI.Prague] EWGI. 2020. EWGI Discussion of P1858R1.
http://wiki.edg.com/bin/view/Wg21prague/P1858R1SG17

[N3728] Mike Spertus. 2013. Packaging Parameter Packs (Rev. 2).
https://wg21.link/n3728

[N4072] Maurice Bos. 2014. Fixed Size Parameter Packs.
https://wg21.link/n4072

[N4191] A. Sutton, R. Smith. 2014. Folding expressions.
https://wg21.link/n4191

[N4235] Daveed Vandevoorde. 2014. Selecting from Parameter Packs.
https://wg21.link/n4235

[P0195R2] Robert Haberlach, Richard Smith. 2016. Pack expansions in *using-declaration*s.
https://wg21.link/p0195r2

[P0341R0] Mike Spertus. 2016. parameter packs outside of templates.
https://wg21.link/p0341r0

[P0535R0] Matthew Woehlke. 2017. Generalized Unpacking and Parameter Pack Slicing.
https://wg21.link/p0535r0

[P0780R2] Barry Revzin. 2018. Allow pack expansion in lambda init-capture.
https://wg21.link/p0780r2

[P1045R1] David Stone. 2019. constexpr Function Parameters.
https://wg21.link/p1045r1

[P1061R0] Barry Revzin, Jonathan Wakely. 2018. Structured Bindings can introduce a Pack.
https://wg21.link/p1061r0

[P1061R1] Barry Revzin, Jonathan Wakely. 2019. Structured Bindings can introduce a Pack.
https://wg21.link/p1061r1

[P1219R1] James Touton. 2019. Homogeneous variadic function parameters.
https://wg21.link/p1219r1

[P1240R0] Andrew Sutton, Faisal Vali, Daveed Vandevoorde. 2018. Scalable Reflection in C++.
https://wg21.link/p1240r0

[P1306R1] Andrew Sutton, Sam Goodrick, Daveed Vandevoorde. 2019. Expansion statements.
https://wg21.link/p1306r1

[P1717R0] Andrew Sutton, Wyatt Childers. 2019. Compile-time Metaprogramming in C++.
https://wg21.link/p1717r0

[P1789R0] Alisdair Meredith. 2019. Library Support for Expansion Statements.
https://wg21.link/p1789r0

[P1858R0] Barry Revzin. 2019. Generalized pack declaration and usage.
https://wg21.link/p1858r0

[P1858R1] Barry Revzin. 2020. Generalized pack declaration and usage.
https://wg21.link/p1858r1

[P2120R0] Barry Revzin. 2020. Simplified structured bindings protocol with pack aliases.
https://wg21.link/p2120r0

[Smith.Pack] Richard Smith. 2013. A problem with generalized lambda captures and pack expansion. https://groups.google.com/a/isocpp.org/d/msg/std-discussion/ePRzn4K7VcM/Cvy8M8EL3YAJ